

Verification Agent 設計規格

文件版本：1.1

日期：2026-06-27（設計） / 2026-07-01（實作完成）

狀態：✅ 已實作，並用 3 組真實公司合約驗證（NDA / 軟體採購 / 軟體維護）

目的：防止 Risk Rule Engine 因語意彈性不足或 Parser 解析失敗而產生漏判，確保高風險條款零遺漏。

2026-07-01 實作紀錄：核心設計（Layer 2 語意補漏、Case A/B/C 交叉核對、clause_id 正規化）已依本規格實作於 `src/services/contract/verifier.py`，並接入 `orchestrator.py` Step 4b。與原始設計的差異與尚未實作項目，見文末「實作差異與待辦」。

一、問題背景

現有 Risk Rule Engine 使用 15 條 Regex / 關鍵字規則，在以下情況會產生漏判：

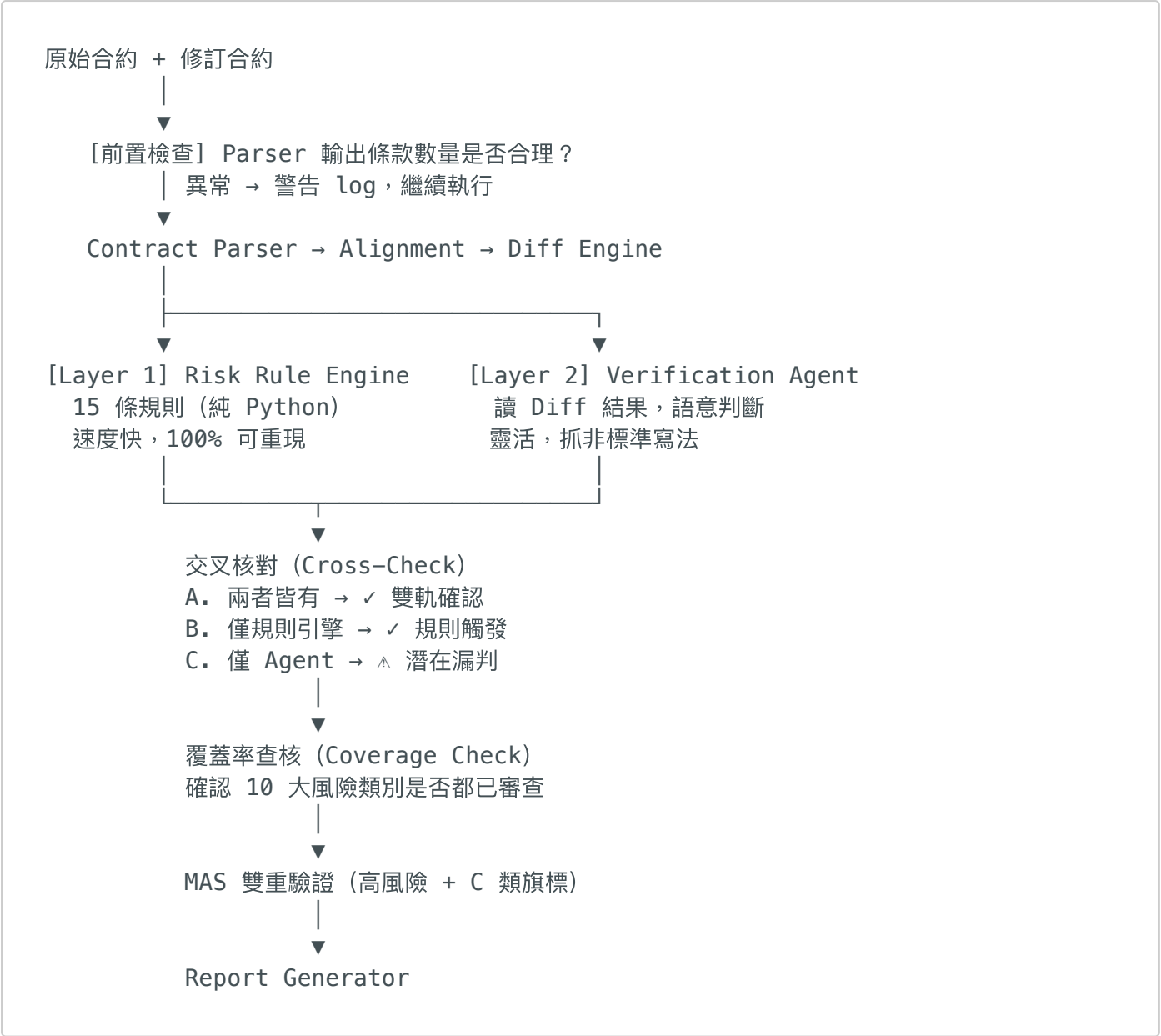
漏判原因	範例
非標準中文寫法	「可用率不低於千分之九百九十五」→ 規則未覆蓋
Parser 解析失敗	條款被切錯，clause_id 標為 "?"，規則比對不上
規則庫未涵蓋的新型風險	乙方加入「AI 訓練資料使用權」等新型條款

任何漏判在法律合約場景都可能導致高損失，因此需要獨立的補漏機制。

二、設計原則

- 補漏不替代：**Verification Agent 是安全網，不是取代 Rule Engine
- 保守優先：**任何一層有旗標就保留，寧可多判不漏判
- 可追溯：**Agent 補漏的旗標使用獨立 risk_code，與規則引擎結果可區分
- 成本控制：**Agent 讀 Diff 結果（不讀全文），避免 context window 超限

三、系統架構



四、Verification Agent 提示詞

VERIFICATION_AGENT_SYSTEM_PROMPT = """你是資深甲方法律顧問與合約審查專家。

任務：

審查以下合約條款的修改內容，找出對甲方（買方）不利的實質風險變更。

評估重點：

- SLA 可用率降低
- 回應或修復時間拉長
- 賠償上限縮水
- 保護條款被刪除
- 保密期縮短
- 智財權移轉至乙方
- 責任方向反轉

- 不可抗力範圍擴大
- 管轄地改變
- 資料控制權喪失

忽略：純字詞修正、排版調整、對甲方有利的修改。

必須輸出 JSON 陣列（不含其他說明文字）：

```
[
  {
    "clause_id": "條文編號或標題",
    "risk_level": "high | medium | low",
    "trigger_reason": "修改差異與不利影響說明",
    "evidence_text": "修改版中的關鍵句子"
  }
]
```

若無任何風險，輸出空陣列：[]
.....

五、交叉核對邏輯

三種情況分類

情況	Rule Engine	Verification Agent	處理	UI 標記
A. 雙軌確認	有	有	正常進入 MAS	✓ 雙軌確認
B. 規則觸發	有	無	信任規則引擎，保留	✓ 規則觸發
C. 補漏警示	無	有	加入報告，MAS 加強驗證	⚠ 潛在漏判

clause_id 正規化（解決對齊問題）

Agent 回傳的 clause_id 格式不一定與 Parser 一致，需正規化後再比對：

```
import re
import difflib

def normalize_clause_id(cid: str) -> str:
```

```

"""將不同格式的 clause_id 正規化為可比對的字串"""
cid = cid.strip()
# 移除「第」「條」「款」等中文前後綴
cid = re.sub(r'^第|條$|款$', '', cid)
# 移除空格
cid = cid.replace(' ', '')
return cid.lower()

def find_matching_diff(agent_clause_id: str, diffs: list) -> object:
    """用模糊比對找到對應的 DiffItem"""
    normalized_agent = normalize_clause_id(agent_clause_id)

    # 先嘗試完全比對
    for d in diffs:
        if normalize_clause_id(str(d.clause_id)) == normalized_agent:
            return d

    # 再嘗試模糊比對 (相似度 > 80%)
    diff_ids = [str(d.clause_id) for d in diffs]
    matches = difflib.get_close_matches(agent_clause_id, diff_ids, n=1,
cutoff=0.8)
    if matches:
        return next(d for d in diffs if str(d.clause_id) == matches[0])

    return None

```

六、覆蓋率查核 (Coverage Check)

在交叉核對之後，自動確認 10 大風險類別是否都已審查：

```

RISK_COVERAGE_CHECKLIST = [
    "SLA 可用率",
    "回應修復時間",
    "賠償上限",
    "保護條款",
    "保密條款",
    "智財權歸屬",
    "責任方向",
    "不可抗力",
    "管轄法院",
    "資料控制權",
]

def check_coverage(all_flags: list) -> dict:
    """
    確認每個風險類別是否已被審查 (有旗標 = 已審查有風險，
    無旗標不等於未審查，而是審查後確認無風險)
    回傳：各類別的審查狀態
    """
    covered = {category: False for category in RISK_COVERAGE_CHECKLIST}
    for flag in all_flags:

```

```
for category in RISK_COVERAGE_CHECKLIST:
    if category in (flag.trigger_reason or ""):
        covered[category] = True
return covered
```

設計意圖：讓系統能明確說「第 X 類已審查，無風險」，而非沉默（沉默可能是漏判或已審查，使用者無法區分）。

七、程式碼結構

新增檔案：[src/services/contract/verifier.py](#)

```
import json
import re
import difflib
import os
from typing import List, Dict, Any, Optional
from .schemas import RiskFlag, DiffItem

RISK_CODE_AGENT = "RISK_AGENT_AUDITED"

class VerificationAgent:
    def __init__(self):
        self.api_key = os.environ.get("GEMINI_API_KEY") or
os.environ.get("ANTHROPIC_API_KEY")

    def audit_diffs(self, diffs: List[DiffItem]) -> List[Dict[str, Any]]:
        """將 Diff 結果送給 LLM，捕獲語意風險。"""
        if not self.api_key or not diffs:
            return []

        diff_text = "\n\n".join([
            f"【條款 {d.clause_id}】\n原文：{d.old_text or '（無）'}\n改後：{d.new_text or '（已刪除）'}"
            for d in diffs if d.change_type != "unchanged"
        ])

        # 呼叫 LLM (Gemini 主 / Claude 備)，解析回傳 JSON
        # ... (實作略，與 llm_service.py 呼叫方式相同)
        return []

    def cross_check_risks(
        engine_flags: List[RiskFlag],
        agent_flags: List[Dict[str, Any]],
        diffs: List[DiffItem]
    ) -> List[RiskFlag]:
        """交叉核對，補充漏判，輸出合併後的 final_flags。"""
        final_flags = list(engine_flags)
        engine_ids = {normalize_clause_id(str(f.clause_id)) for f in
engine_flags}
```

```

for af in agent_flags:
    normalized = normalize_clause_id(af["clause_id"])
    if normalized in engine_ids:
        continue # Case A/B: 規則引擎已有，略過

    # Case C: 補漏旗標
    matched = find_matching_diff(af["clause_id"], diffs)
    fallback = RiskFlag(
        clause_id=af["clause_id"],
        risk_code=RISK_CODE_AGENT,
        risk_level=af["risk_level"],
        risk_direction="adverse",
        trigger_reason=f"【Agent 補漏】{af['trigger_reason']}",
        old_text=matched.old_text if matched else "",
        new_text=matched.new_text if matched else
af.get("evidence_text", ""),
        change_type=matched.change_type if matched else "modified",
        mas_status="pending", # C 類旗標強制進入 MAS 加強驗證
    )
    final_flags.append(fallback)

return final_flags

def normalize_clause_id(cid: str) -> str:
    cid = re.sub(r'^第|條$|款$', '', cid.strip())
    return cid.replace(' ', '').lower()

def find_matching_diff(agent_clause_id: str, diffs: List[DiffItem]) ->
Optional[DiffItem]:
    normalized = normalize_clause_id(agent_clause_id)
    for d in diffs:
        if normalize_clause_id(str(d.clause_id)) == normalized:
            return d
    ids = [str(d.clause_id) for d in diffs]
    matches = difflib.get_close_matches(agent_clause_id, ids, n=1,
cutoff=0.8)
    return next((d for d in diffs if str(d.clause_id) == matches[0]), None)
if matches else None

```

修改 `orchestrator.py` : 在 `risk_engine` 之後、`mas_service` 之前插入：

```

# 現有流程
engine_flags = risk_engine.analyze(diff_items)

# 新增: Verification Agent 補漏
agent = VerificationAgent()
agent_flags = agent.audit_diffs(diff_items)
all_flags = cross_check_risks(engine_flags, agent_flags, diff_items)

# 繼續原有流程
mas_results = mas_service.validate(all_flags)

```

八、Parser 前置健康檢查

```
def check_parser_health(clauses: list, contract_text: str) -> bool:
    """
    簡單確認 Parser 輸出是否合理。
    若條款數量遠少於文字行數，可能是 Parser 切割失敗。
    """
    line_count = contract_text.count('\n')
    clause_count = len(clauses)

    if clause_count == 0:
        log.warning("Parser 輸出 0 條條款，疑似解析失敗")
        return False
    if clause_count == 1 and line_count > 50:
        log.warning(f"Parser 輸出僅 1 條條款，但原文有 {line_count} 行，疑似 DOCX 切割錯誤")
        return False
    return True
```

九、實作優先順序

項目	工作量	建議時機
verifier.py 骨架 + cross_check 邏輯	1 天	法務合約拿到後立即做
LLM 呼叫實作（接 Gemini/Claude）	0.5 天	同上
clause_id 正規化 + 模糊比對	0.5 天	同上
Parser 健康檢查	0.5 天	同上
覆蓋率查核	0.5 天	可延後
UI 標記（⚠ 潛在漏判）	0.5 天	可延後

十、與現有 MAS 的關係

現有 MAS (Phase 1.5) :

Rule Engine → 高風險旗標 → Agent A / B 評估 → confirmed / pending

加入 Verification Agent 後：

RuleEngine + Verification Agent → 合併旗標（含 Case C）

→ 所有旗標進入 Agent A / B
→ C 類旗標（補漏）強制標為 pending，需人工複核

Verification Agent 不是 MAS 的一部分，而是在 MAS 之前的「旗標擴充層」。

十一、實作差異與待辦（2026-07-01）

與原始設計不同之處

- **候選規則紀錄（新增，原規格沒有）**：Case C 補漏旗標不會自動生成 Rule Engine 規則（自動生成有風險——沒人審查過的 regex 可能誤判所有未來合約）。改為每筆 Case C 追加寫入 `candidate_rules.jsonl`，包含 `category_guess`（比對既有風險類別關鍵字，或標「新類別候選」）。用真實合約測試時，累積 30 筆裡有 5 筆都歸類「違約金/罰則」，證明這個機制能有效指出「哪一類該考慮升級成正式規則」。
- **risk_level 防呆**：LLM 回傳的 risk_level 偶爾不是 `high/medium/low`（例如中文「高」），已加 `normalize_risk_level()` 做 clamp，避免不合法值讓旗標從整體評估統計中靜默消失。
- **Case C 去重**：LLM 對同一條款重複產生補漏旗標時（幻覺常見情形），已加 `seen_agent_ids` 去重，只保留第一筆。

尚未實作（規格已寫，程式碼未做）

- **第六節 覆蓋率查核（Coverage Check）**：`check_coverage()` 尚未接入 pipeline。
- **第八節 Parser 前置健康檢查**：`check_parser_health()` 尚未接入 pipeline。

用真實合約測試時發現、規格未預期到的問題

- **Verification Agent 只讀 diffs，不讀全文**——這是刻意設計（控制 context window），但代表如果 Step 3（Diff Engine）本身就漏掉某段內容（見下一項），Verification Agent 完全看不到，補漏不了。
- **Diff Engine 涵蓋率漏洞（比 Verification Agent 更上游）**：純新增/刪除條款若沒有可辨識的條號，會被 `diff_engine.py` 直接丟棄，不會進入 diffs 清單。真實採購合約測出 138 段因此消失（已修復，補上跟 modified 型態一樣 title fallback）。

- **Verification Agent 的交叉核對邏輯 (Case A/B/C)** 曾經只比對 `clause_id`，沒有比對「風險內容是否為同一件事」(✅ 2026-07-02 已修復)：若同一條款同時存在規則引擎抓到的風險 A 與 Agent 抓到的風險 B (不同風險維度)，舊邏輯會因為 `clause_id` 重複而把 Agent 的風險 B 整個丟棄。用合成的 v6 Demo 範例 (軟體維護合約，違約金費率 0.3%→千分之一 + 上限 50%→30%) 實測時發現，這個 bug 是否觸發取決於 Agent 當次把 `clause_id` 標成什麼字串——同一份合約重跑，這個發現有時活下來、有時被吞掉，屬於非決定性風險。修法：比對 key 從單純 `clause_id` 改成 `(clause_id, 風險類別)`，兩邊都用同一套 `categorize_candidate()` 分類 (規則引擎透過新增的 `RISK_CODE_CATEGORY` 對照表換算)，只有「同條款且同類別」才視為重複丟棄。v6 重跑 3 次結果穩定一致，3 份真實合約 + v2-v5 回歸測試皆正常。**已知殘留限制**：關鍵字分類本身仍有歧義 (例如「違約金上限」同時命中「違約金/罰則」與「賠償上限」兩類)，最壞情況是多出一筆無害的重複 Case C 旗標，不會再導致真正的發現被吞掉——符合「寧可多判不漏判」原則，不再深究。
- **Verification Agent 本身非決定性**：同一份合約重跑，Case C 補漏數量會浮動 (真實測試中同一份採購合約在 6~8 項之間浮動)，代表補漏覆蓋率是機率性的，不是保證的。目前沒有讓使用者知道這件事的機制。

知識庫外部化 (2026-07-02，新增，原規格沒有)

`VERIFICATION_SYSTEM_PROMPT` 原本把「評估重點」清單寫死在 `verifier.py` 的 Python 字串裡。比照 `mas_service.py` 讓 MAS Agent A/B 從 `.claude/skills/` 動態載入知識庫的既有模式，`verifier.py` 新增同款 `_load_skill_section()`，改從 `contract-risk-analysis.md` 的「## Verification Agent 知識庫 (語意補漏審查員)」section 讀取評估重點。

這不是架構一致性的形式主義：這批 `.md` 檔案雖然也是 Claude Code 開發時用的 skill，但執行期讀取純粹是 Python 檔案 I/O，跟 Claude Code 完全無依賴關係。真正的價值是讓非工程背景的法務同仁能直接編輯這份知識庫，不需要碰程式碼——這件事在法務祐銓答應以顧問角色協助審查系統之後，變成一個實際可用的協作管道，不只是理論上的好處。找不到 skill section 時優雅降級為簡短的通用指示 (不拋出例外)，已測試驗證。